HỌC VIỆN STARTUP – AI5.VN

CHƯƠNG TRÌNH ĐÀO TẠO FULLSTACK / DEVSECOPS

ĐỀ BÀI TẬP TEAMWORK
THỰC CHIẾN DOANH NGHIỆP

TECHBYTE SERVICE DESK

Xây dựng, triển khai và vận hành hệ thống hỗ trợ kỹ thuật

THÔNG TIN ĐỀ BÀI	NỘI DUNG
Hình thức	Teamwork 05 thành viên; mỗi cá nhân có phần việc độc lập và nghiệm thu chung.
Phạm vi	Frontend, Backend, Database, GitHub, Docker, CI/CD, Nginx, Cloudflare, vận hành & xử lý sự cố.
Yêu cầu hạ tầng	01 VPS Ubuntu + tên miền; Docker và Docker Compose; GitHub repository.
Đầu ra	Hệ thống truy cập được qua HTTPS, mã nguồn, thiết kế, pipeline, bằng chứng và báo cáo.
Chính sách AI	Được phép sử dụng AI, nhưng bắt buộc ghi nhận và giải thích được kết quả.

Hà Nội, tháng 11 năm 2026

Tài liệu dùng để giao bài, tổ chức thực hành và đánh giá học viên.

I. MỤC ĐÍCH, YÊU CẦU CHUNG

Từ bài teamwork 02 tiếng với các nội dung Git/GitHub, Docker, Docker Compose và CI/CD cơ bản, nhóm được giao bài toán mở rộng theo quy trình phát triển phần mềm doanh nghiệp: có yêu cầu nghiệp vụ, thiết kế hệ thống, phát triển các thành phần độc lập, tích hợp, phát hành và bảo trì. Mục tiêu là học viên biết làm việc cùng nhau, tự chịu trách nhiệm phần được giao và chứng minh hệ thống vận hành bằng kết quả thực tế.

1. Mục tiêu năng lực

- Hiểu và thực hiện quy trình: nhận yêu cầu → phân tích → thiết kế → phân công → phát triển → kiểm thử → CI/CD → triển khai → nghiệm thu → bàn giao.
- Phân biệt các vai trò Frontend, Backend, Database, DevOps và PM/System Architect; biết phối hợp khi thay đổi API, schema hoặc cấu hình triển khai.
- Thiết kế được luồng request, luồng triển khai và quan hệ dữ liệu; không chỉ dựng một sơ đồ để minh họa.
- Triển khai ứng dụng chạy thật qua tên miền sử dụng Cloudflare, Nginx, Docker Compose và HTTPS.
- Biết xác định hiện tượng, khoanh vùng nguyên nhân, xử lý sự cố, xác nhận lại kết quả và tránh tái diễn.
- Sử dụng AI để học, tra cứu, hỗ trợ viết code và kiểm thử có trách nhiệm; bảo vệ được từng quyết định kỹ thuật.

2. Nguyên tắc thực hiện

- Tổ chức đúng 05 thành viên. Mỗi người phải có đầu ra riêng, lịch sử commit/PR và phần bảo vệ kỹ thuật.
- Không yêu cầu một công cụ duy nhất: nhóm được lựa chọn công nghệ trong phạm vi cho phép nhưng phải giải thích lý do.
- Không lưu mật khẩu, token, private key trong repository; không mở database công khai trên Internet.
- Không tự ý xóa database/volume hoặc chạy thử nghiệm phá hoại trên môi trường có dữ liệu thật. Chỉ sử dụng dữ liệu giả lập và môi trường lab.
- Nộp đầy đủ sản phẩm, tài liệu, bằng chứng; lời khẳng định “đã chạy được” không thay cho kết quả kiểm tra.

II. BỐI CẢNH VÀ PHẠM VI BÀI TOÁN

Một công ty công nghệ cung cấp dịch vụ xây dựng và vận hành website, API cho khách hàng. Hiện nay, yêu cầu hỗ trợ được tiếp nhận qua các kênh rời rạc nên khó kiểm soát người xử lý, thời gian phản hồi và lịch sử sự cố. Ban giám đốc giao nhóm xây dựng hệ thống “TechByte Service Desk” để chuẩn hóa quy trình tiếp nhận và xử lý yêu cầu hỗ trợ kỹ thuật.

1. Người dùng và luồng nghiệp vụ

1. Khách hàng hoặc nhân viên đăng nhập, tạo ticket mô tả sự cố, chọn mức ưu tiên và xem tiến độ.
2. Kỹ thuật viên xem danh sách ticket được giao, cập nhật trạng thái, bình luận và ghi nhận kết quả xử lý.
3. Quản trị viên hoặc người quản lý xem dashboard, phân công người xử lý và theo dõi lịch sử thay đổi.
4. Hệ thống lưu được ai tạo, ai xử lý, thay đổi trạng thái lúc nào và nội dung trao đổi liên quan.

2. Phạm vi bắt buộc và phần nâng cao

Mức	Nội dung	Tiêu chí kết thúc
Bắt buộc (P0)	Đăng nhập; ticket CRUD theo quyền; bình luận; lịch sử trạng thái; dashboard; PostgreSQL; Docker Compose; Git workflow; CI/CD; Cloudflare HTTPS.	Có thể thao tác đầy đủ từ trình duyệt và API; thay đổi phiên bản có kiểm chứng.
Khuyến khích (P1)	Tìm kiếm, phân trang, lọc ưu tiên; audit log; email thông báo giả lập; monitoring; staging.	Có bằng chứng thực tế và không làm hỏng chức năng P0.
Mở rộng (P2)	Tách 02 VPS; triển khai không gián đoạn ở mức cơ bản; đo load test; báo cáo thống kê SLA.	Nhóm giải thích được trade-off về chi phí, vận hành và độ phức tạp.

III. CƠ CẤU NHÓM VÀ GIAO VIỆC

PM không được viết thay toàn bộ bài. Mọi thành viên tham gia họp thiết kế, thống nhất API Contract/Database Schema và bảo vệ sản phẩm cuối kỳ.

Vị trí	Nhiệm vụ cụ thể	Đầu ra phải nộp
01. PM / System Architect / QA	Phân tích yêu cầu, quản trị backlog, phân task, sơ đồ kiến trúc, API Contract, quản lý PR, test tích hợp, nghiệm thu.	Yêu cầu nghiệp vụ, backlog, 02 sơ đồ, test report, biên bản họp, tối thiểu 02 PR thực chất.
02. Frontend Developer	Giao diện đăng nhập, danh sách, tạo/chi tiết ticket, dashboard, trạng thái lỗi và loading, gọi API.	FE source, các trang có thể sử dụng, screenshots, PR và hướng dẫn chạy local.
03. Backend Developer	API, xác thực và phân quyền, nghiệp vụ ticket, validation, health/readiness, kiểm thử tự động.	API source, OpenAPI/Postman, kết quả test, PR và log hợp lệ.
04. Database Engineer	ERD, schema, constraints, index, migration, seed, backup/restore; phối hợp với BE.	ERD, SQL/migration, data dictionary, kịch bản backup/restore và evidence.
05. DevOps / Cloud Engineer	Dockerfiles, Compose, Nginx, server, domain/Cloudflare, CI/CD, logging, rollback, cấu hình an toàn.	Compose, workflows, cấu hình proxy, sơ đồ mạng, runbook và deployment evidence.

1. Quy định phối hợp

- Ngày đầu tiên, nhóm phải thống nhất interface giữa các thành phần: đường dẫn API, cấu trúc JSON, mã lỗi, biến môi trường, thiết kế bảng dữ liệu.
- Backend thay đổi API phải cập nhật contract và thông báo Frontend; Database thay đổi schema phải có migration và thông báo Backend.
- PM phải thực hiện review liên quan đến tính đúng yêu cầu, DevOps kiểm tra khả năng build/deploy, mỗi PR có ít nhất 01 người review khác tác giả.
- Mọi lỗi tích hợp phải được ghi thành issue: hiện tượng, người phụ trách, mức ưu tiên, hướng xử lý và kết quả xác minh.

IV. YÊU CẦU CHỨC NĂNG ỨNG DỤNG

1. Frontend

- Trang đăng nhập: nhập tài khoản/mật khẩu; hiển thị lỗi đúng khi thông tin không hợp lệ; có logout.
- Dashboard: tổng số ticket, số đang mở, đang xử lý và hoàn thành; lấy từ API, không hard-code.
- Danh sách ticket: cột mã, tiêu đề, ưu tiên, trạng thái, người phụ trách, thời gian cập nhật; có tìm kiếm/lọc trạng thái.
- Tạo ticket: tiêu đề, mô tả, mức ưu tiên; validation trước khi gửi; hiển thị thông báo sau thao tác.
- Chi tiết ticket: thông tin, bình luận, lịch sử thay đổi trạng thái, cập nhật trạng thái theo quyền.
- Có loading/empty/error states; không để API lỗi làm giao diện trắng hoặc đứng.

2. Backend và phân quyền

- Xác thực bằng phiên hoặc token theo phương án nhóm chọn; password phải hash, không lưu dạng rõ.
- Tối thiểu hai nhóm quyền: STAFF (tạo/xem ticket thuộc phạm vi được phép) và ADMIN (quản lý/phân công); đề xuất thêm TECHNICIAN nếu đủ thời gian.
- Mọi thao tác cập nhật phải kiểm tra quyền ở backend, không chỉ ẩn nút trên frontend.
- Trạng thái ticket tối thiểu: OPEN → IN_PROGRESS → RESOLVED → CLOSED; nhóm phải xác định các chuyển đổi hợp lệ.
- API xử lý dữ liệu nhập không hợp lệ bằng mã lỗi đúng; không để lộ stacktrace hoặc secrets.
- Cung cấp health endpoint và kiểm tra kết nối DB trong readiness phù hợp để phát hiện lỗi sau deploy.

3. Danh sách API tối thiểu

Phương thức	Endpoint tham khảo	Ý nghĩa / kiểm tra
POST	/api/auth/login	Đăng nhập; sai thông tin trả lỗi hợp lý; không trả

		password.
GET	/api/me	Thông tin người dùng hiện tại.
GET	/api/tickets	Danh sách ticket, hỗ trợ filter; bảo đảm phạm vi truy cập.
POST	/api/tickets	Tạo ticket; kiểm tra validation.
GET	/api/tickets/{id}	Chi tiết ticket; kiểm tra quyền truy cập.
PATCH	/api/tickets/{id}/status	Đổi trạng thái; ghi lịch sử.
POST	/api/tickets/{id}/comments	Gửi bình luận; lưu người gửi, thời gian.
GET	/api/dashboard/summary	Thống kê thực từ DB.
GET	/health và /ready	Trạng thái tiến trình và khả năng phục vụ.

Các endpoint trên là hợp đồng tham khảo để giao việc; nhóm có thể chỉnh tên nhưng phải thống nhất trước ngày nghiệm thu đầu tiên và cập nhật tài liệu OpenAPI/Postman.

4. Bộ dữ liệu mẫu và trường hợp kiểm thử

- Tạo tối thiểu 08 người dùng giả lập, 20 ticket thuộc nhiều trạng thái và 10 bình luận. Không sử dụng dữ liệu khách hàng thật.
- Kiểm thử một người dùng không được sửa/xem ticket ngoài phạm vi quyền của mình.
- Kiểm thử input thiếu tiêu đề, mô tả quá dài, ticket ID không tồn tại, thao tác cập nhật trái phép.
- Kiểm thử đăng nhập sai, token hết hạn hoặc chưa đăng nhập và kết quả trả về phù hợp.
- Kiểm thử dashboard thay đổi theo dữ liệu thực, không sử dụng số liệu gắn cứng trong code.

V. THIẾT KẾ CƠ SỞ DỮ LIỆU

Nhóm Database Engineer phải làm việc với Backend Developer để xây dựng mô hình có khả năng phục vụ các luồng nghiệp vụ trong Mục IV; tuyệt đối không tạo các bảng chỉ để đủ số lượng.

Bảng	**Trường chính gợi ý**	**Yêu cầu kỹ thuật**
users	id, email, password_hash, full_name, role, created_at	Email duy nhất; mật khẩu lưu hash; có vai trò.
tickets	id, code, title, description, priority, status, created_by, assigned_to, created_at, updated_at	Khóa ngoại tới users; có index cho trạng thái và ngày.
ticket_comments	id, ticket_id, author_id, content, created_at	FK hợp lệ; không bình luận vào ticket không tồn tại.
ticket_history	id, ticket_id, actor_id, old_status, new_status, changed_at	Lưu lịch sử trạng thái để truy vết.

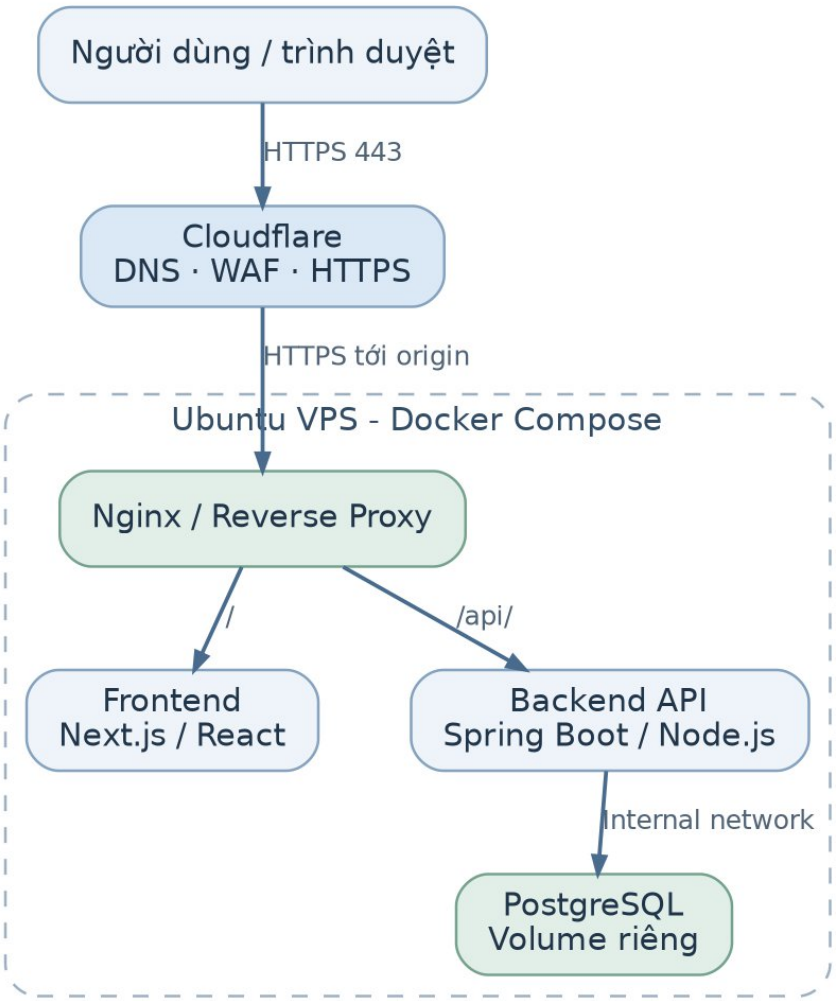
- Nộp ERD thể hiện đúng PK/FK, quan hệ 1-n, nullability và các ràng buộc chính.
- Mọi thay đổi schema phải đi qua migration có version; hạn chế chỉnh bảng thủ công trên server.

- Docker Compose sử dụng named volume hoặc volume quản lý phù hợp để dữ liệu không mất khi restart/recreate container.
- Thực hiện backup và restore trên dữ liệu giả lập; lập biên bản kết quả, không xóa hoặc ghi đè dữ liệu thật.
- Tài khoản kết nối DB chỉ được cấp quyền cần thiết; không đặt database public hoặc commit credential vào Git.

VI. SYSTEM DESIGN VÀ HẠ TẦNG TRIỂN KHAI

1. Kiến trúc tham chiếu

Môi trường bắt buộc: 01 Ubuntu VPS, Docker Engine + Compose, một tên miền/subdomain được cấu hình qua Cloudflare. Stack đề xuất: Next.js/React, Java Spring Boot hoặc Node.js, PostgreSQL và Nginx. Nhóm phải công bố phiên bản công nghệ đã chọn trong README.



Hình 1. Luồng người dùng, proxy, ứng dụng và dữ liệu; nhóm phải vẽ lại theo triển khai thực tế.

2. Yêu cầu thiết kế bắt buộc

- Phân biệt công khai và nội bộ: chỉ mở cổng thực sự phục vụ truy cập từ Internet; API/DB không mở public khi không cần thiết.
- Nginx định tuyến / tới Frontend và /api tới Backend; sử dụng tên service hoặc địa chỉ hợp lệ trong network của Compose.
- Tách biến môi trường phục vụ local/staging/production và không commit file chứa giá trị bí mật.
- Cloudflare sử dụng DNS proxy phù hợp, SSL/TLS Full (strict) với chứng chỉ origin hợp lệ; nêu rõ đường đi HTTPS.
- Bổ sung Docker volume cho database, healthcheck, restart policy và log rotation theo mức phù hợp.
- Yêu cầu 02 sơ đồ riêng: (1) kiến trúc request và mạng; (2) source → CI/CD → registry → VPS; có chú thích port, service, network.

3. Hạ tầng tham khảo

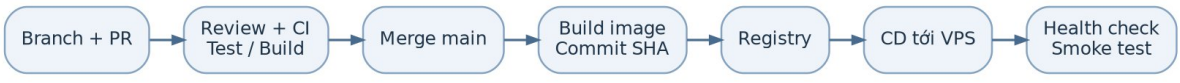
Thành phần	Yêu cầu / lưu ý
Ubuntu VPS	Gợi ý 04 vCPU, 08 GB RAM để chạy đồng thời FE, BE Java và DB; cấu hình khác cần tự đánh giá tải.
Domain	Tên miền hoặc subdomain của nhóm; Cloudflare quản lý DNS và TLS.
Docker Compose	Ít nhất frontend, backend, database, reverse-proxy; network nội bộ phù hợp; volume riêng cho DB.
Bảo mật	Firewall, SSH theo IP/quy định lab; không expose DB 5432; không nhúng credentials trong YAML.
Môi trường	Tối thiểu local + production (lab). Staging là điểm cộng nếu tách được cấu hình.

VII. GIT, CI/CD VÀ QUY TRÌNH RELEASE

1. Git workflow bắt buộc

- Một repo chung; branch main được bảo vệ bằng quy định nhóm, không push trực tiếp.
- Mỗi thành viên tối thiểu 02 PR có giá trị; PR có nội dung, ảnh hoặc test evidence, người review và kết quả xử lý góp ý.
- Quy ước branch ví dụ: feature/frontend-ticket, feature/backend-ticket, feature/db-migration, feature/compose-ci, docs/system-design.
- Trường hợp conflict: người tạo PR phải xử lý và giải thích cách giữ các thay đổi hợp lệ.

2. Pipeline chuẩn



Hình 2. Luồng phát hành có review, test, build image, deploy và kiểm tra sau triển khai.

3. Yêu cầu CI

- Khi có Pull Request: lint/format nếu có; chạy kiểm thử Backend, build Frontend/Backend; báo kết quả pass/fail.
- Khi merge vào main: tạo image gắn commit SHA hoặc version; push registry; không phụ thuộc tag latest để rollback.
- Secret của registry/SSH được cấu hình bằng cơ chế secrets có quyền tối thiểu; không in secrets ra log.

4. Yêu cầu CD và rollback

- Đưa image phiên bản đã kiểm thử lên VPS, cập nhật Compose, chạy migration theo quy trình nhóm đã thống nhất.
- Sau deploy: kiểm tra trạng thái container, health/readiness, API thật, thao tác FE từ domain và log 5xx.
- Nếu smoke test thất bại: có quy trình khôi phục image/config phiên bản trước; riêng migration không tương thích phải có phương án riêng.
- Ghi lại thông tin release: phiên bản, người triển khai, thời gian, kết quả kiểm tra và thao tác rollback nếu có.

```
docker compose config -q
 docker compose ps
 docker compose logs --tail=100 backend
 curl -fsS http://127.0.0.1:<PORT>/health
 curl -I https://<DOMAIN>
```

Các lệnh chỉ là mẫu minh họa. Học viên phải thay placeholder theo hệ thống thực tế, giải thích ý nghĩa và kết quả trả về.

VIII. CLOUDFLARE, KIỂM SOÁT TRUY CẬP VÀ CẤU HÌNH AN TOÀN

Yêu cầu nhóm triển khai Cloudflare theo cách sử dụng thực tế, không chỉ trỏ bản ghi DNS rồi kết luận đã cấu hình xong.

- Trỏ bản ghi DNS của tên miền tới IP VPS; xác nhận truy cập qua HTTPS từ Internet.
- Sử dụng SSL/TLS Full (strict) và kiểm chứng origin có chứng chỉ hợp lệ; không dùng Flexible để chữa lỗi cấu hình server.
- Bật proxy cho hostname được bảo vệ; cấu hình WAF/Security Rule ở mức phù hợp với nhu cầu và gói Cloudflare hiện có.
- Có thử nghiệm hạn chế truy cập một endpoint nhạy cảm hoặc một IP thử nghiệm mà không khóa nhầm toàn hệ thống.
- Cấu hình cache đúng đối tượng; không cache API chứa dữ liệu cá nhân, phản hồi đăng nhập hoặc dữ liệu phụ thuộc cookie/token.

- Nêu cách ngăn người dùng bỏ qua Cloudflare để truy cập trực tiếp Origin khi mô hình cho phép; không công khai dịch vụ nội bộ.
- Có ghi nhận quy tắc tạo mới, trạng thái trước/sau thay đổi và phương án rollback rule.

IX. 05 TÌNH HUỐNG SỰ CỐ ĐỂ NGHIỆM THU

Giảng viên sẽ lựa chọn một hoặc nhiều tình huống để đánh giá trực tiếp. Học viên chỉ được thao tác trên môi trường được phép và phải ghi nhận bằng chứng trước/sau. Đề không yêu cầu phá hoại hệ thống hay xóa dữ liệu thật.

Tình huống	Biểu hiện được giao	Đầu ra cần chứng minh
01. Reverse Proxy 502	Frontend truy cập được nhưng /api trả 502 sau thay đổi cấu hình.	Sơ đồ request, giả thuyết, các lệnh kiểm tra, nguyên nhân, cấu hình sửa, smoke test.
02. Release lỗi	CI xanh, container chạy nhưng API đổi trạng thái ticket lỗi sau phiên bản mới.	So sánh release, logs/test, xác định phạm vi ảnh hưởng, rollback, kết luận.
03. DB mất kết nối	API không thể đọc/ghi ticket sau restart service DB.	Phân biệt DB readiness, network, credentials, migration; không xóa volume để chữa cháy.
04. HTTPS/Cloudflare	Domain không truy cập ổn định; origin có thể vẫn phản hồi local.	Khoanh vùng DNS, TLS, firewall, origin, reverse proxy; bằng chứng sửa đúng tầng.
05. Pipeline thất bại	PR hoặc deploy workflow failed.	Xác định đúng job/step, xử lý lỗi, chạy lại, xác nhận image/deploy có version.

Mẫu báo cáo sự cố cho từng tình huống

1. Thời điểm, phiên bản triển khai, triệu chứng và mức ảnh hưởng.
2. Các giả thuyết nguyên nhân (ít nhất 02) và căn cứ kiểm tra.
3. Lệnh/query/log/ảnh đã dùng và kết quả quan sát.
4. Root cause đã xác định hoặc giới hạn kết luận còn chưa chắc chắn.
5. Biện pháp xử lý, kiểm tra sau sửa, thay đổi cần đưa về repository.
6. Đề xuất biện pháp tránh lặp lại lỗi.

X. QUY ĐỊNH SỬ DỤNG AI TRONG TEAMWORK

Nhóm được phép sử dụng AI trong các công đoạn phân tích, thiết kế, viết code, tra cứu lỗi, soạn tài liệu và tạo test cases. Tuy nhiên, công cụ AI không phải người chịu trách nhiệm về kết quả bài làm.

- Tạo AI_USAGE.md gồm: mục tiêu hỏi AI, ý tưởng/đoạn code được sử dụng, người kiểm chứng và kết quả sau kiểm chứng.
- Không cung cấp token, password, SSH key, dữ liệu khách hàng thật hoặc thông tin bí mật lên hệ thống AI.

- Mọi đoạn code do AI đề xuất phải qua review, build/test và được người phụ trách giải thích về logic, rủi ro và phương án thay thế.
- Giảng viên có quyền hỏi bất kỳ thành viên nào về API, database, Docker, Cloudflare, CI/CD và đề nghị thay đổi nhỏ để xác nhận hiểu biết.
- Không chấm cao bài chỉ có file cấu hình/ảnh chụp nhưng thành viên không hiểu luồng vận hành hoặc không tái hiện được kết quả.

XI. HỒ SƠ BÀN GIAO BẮT BUỘC

01. Link GitHub repository, lịch sử branches/commits/PR review và danh sách thành viên.

02. Link tên miền hoạt động; tài khoản demo theo từng vai trò (chỉ trao đổi qua kênh riêng, không ghi password vào báo cáo công khai).

03. README: mục tiêu, stack, kiến trúc, cách chạy local, cách build và deploy.

04. Sơ đồ kiến trúc luồng request; sơ đồ CI/CD; ERD + data dictionary.

05. Tài liệu API contract hoặc OpenAPI/Postman collection; danh sách test cases và kết quả.

06. Dockerfiles, docker-compose.yml, Nginx config, workflows, env.example (không có secret).

07. Evidence: HTTPS/domain, container status, health/API, CI/CD, database persistence và backup/restore lab.

08. Incident reports cho tình huống đã xử lý và release/rollback runbook.

09. AI_USAGE.md và bảng phân công công việc/đánh giá đóng góp cá nhân.

10. Báo cáo nhóm PDF/Word ngắn gọn 5–10 trang, nêu rõ những gì đã làm được/chưa làm được và lý do.

Cấu trúc repository khuyến nghị

```
service-desk/
  frontend/                # Next.js / React
  backend/                 # Spring Boot / Node.js
  database/migrations/     # Phiên bản thay đổi schema
  infra/nginx/             # Reverse proxy
  infra/compose/           # Compose và env.example
  .github/workflows/       # CI + CD
  docs/architecture/       # System Design + ERD
  docs/incidents/          # Báo cáo lỗi, bằng chứng
  docs/api/                # OpenAPI / collection
  AI_USAGE.md
  README.md
```